Mục lục

Phần 1. Tổng quan và tư tưởng thiết kế kiến trúc 1
1.1 Bối cảnh đề tài và lựa chọn dự án thử nghiệm 1
1.2. Bài toán đặt ra đối với dự án cũ 1
1.3. Khái quát quy trình phát triển phần mềm và giới thiệu công cụ nền tảng 2
1.4. Tư tưởng thiết kế hệ thống 3
1.5. Mô hình Kiến trúc 3
Phần 2. Nền tảng quy trình và các công cụ 4
2.1. Git và pre-commit framework 4
2.2. GitLeaks – công cụ rà quét bí mật 4
2.3. PMD và SpotBugs 5
2.3.1 PMD 5
2.3.2 SpotBugs 6
Phần 3. Thực nghiệm quét toàn diện và phân tích 7
3.1. Tool quét audit: audit-scan.bat 7
3.2. Phân tích thực tế mã nguồn 8
3.2.1 GitLeaks 8
3.2.2 PMD 8
3.2.3 SpotBugs + plugin FindSecBugs: 9
3.3. Khoanh vùng nợ kỹ thuật và thiết lập baseline 9
Phần 4. Triển khai quy trình 10
4.1. Tool cài đặt môi trường một chạm setup-env.bat 10
4.2. Thiết lập Quality Gate 10
Phần 5. Tổng kết 11

Phần 1. Tổng quan và tư tưởng thiết kế kiến trúc

1.1 Bối cảnh đề tài và lựa chọn dự án thử nghiệm

Trong khuôn khổ môn học an toàn bảo mật hệ thống thông tin, mục tiêu cốt lõi của nhóm là nghiên cứu và xây dựng một quy trình phát triển phần mềm an toàn (DevSecOps). Để biến lý thuyết thành thực tiễn, quy trình này đòi hỏi một môi trường thực tế để cài đặt, thử nghiệm và đánh giá. Thay vì tạo ra một dự án mới hoàn toàn trống rỗng, nhóm quyết định sử dụng microShop [1] – một dự án website bán tài khoản game online và Steam account.

Sự lựa chọn này hoàn toàn có chủ đích và phản ánh đúng bài toán thực tế của ngành công nghiệp phần mềm. Trên thực tế, các kỹ sư DevSecOps hiếm khi có cơ hội áp dụng quy trình chuẩn chỉ ngay từ ngày đầu tiên của một dự án. Thay vào đó, họ thường xuyên phải tiếp quản các hệ thống đang vận hành, tiến hành kiểm toán sự an toàn của mã nguồn cũ, và khéo léo cài đặt các chốt chặn bảo mật ngay giữa chu kỳ phát triển mà không làm sập hệ thống. microShop chính là "thao trường" lý tưởng nhất để mô phỏng kịch bản tiếp quản và cải tiến này.

1.2. Bài toán đặt ra đối với dự án cũ

Việc tích hợp quy trình DevSecOps vào dự án như microShop đặt ra những thách thức đặc thù. Do mã nguồn này được phát triển từ giai đoạn trước khi các tiêu chuẩn về an toàn bảo mật được áp dụng, bên trong dự án tồn tại một lượng lớn các điểm yếu tiềm tàng và cấu hình lỏng lẻo hay gọi chung là nợ kỹ thuật.

Nếu áp dụng các quy tắc bảo mật một cách cứng nhắc ngay từ đầu, chẳng hạn như cấu hình từ chối mọi đoạn mã có cảnh báo thì tiến độ phát triển sẽ bị đình trệ hoàn toàn. Đội ngũ lập trình viên sẽ liên tục vấp phải các rào cản từ lỗi trong quá khứ, dẫn đến việc không thể hoàn thành các tính năng mới. Do đó, bài toán cốt lõi là phải thiết lập một tuyến phòng thủ vừa đảm bảo nâng cao mức độ an toàn của sản phẩm đầu ra, vừa duy trì được tính linh hoạt và không gây đứt gãy mạch làm việc hàng ngày của nhóm.

[1] Dự án microShop được phát triển bởi nhóm sinh viên: Nguyễn Hải Hưng, Ngô Thanh Hải, Dương Đăng Khoa, Bùi Tuấn Thành, Trần Tuấn Anh. Khóa D23 CNTT PTIT Hà Nội

1.3. Khái quát quy trình phát triển phần mềm và giới thiệu công cụ nền tảng

Để độc giả dễ dàng hình dung cách hệ thống DevSecOps can thiệp vào công việc, cần làm rõ quy trình làm việc thông thường của một lập trình viên cùng các thuật ngữ nền tảng:

1. Quản lý mã nguồn và lệnh commit: Git là công cụ giúp lưu lại toàn bộ lịch sử thay đổi của mã nguồn. Khi lập trình viên viết xong một đoạn code, họ dùng lệnh commit để "đóng gói" và lưu lại những thay đổi này. Git cung cấp một cơ chế gọi là hooks - các kịch bản tự động kích hoạt theo sự kiện. Quy trình này sử dụng pre-commit hooks, đóng vai trò là màng lọc kiểm soát chất lượng tại nguồn. Nó thiết lập một rào cản kỹ thuật ngay tại thời điểm lập trình viên khởi tạo lệnh commit; nếu mã nguồn không đáp ứng được các tiêu chuẩn an toàn đã định nghĩa, hệ thống sẽ tự động từ chối ghi nhận thay đổi. Điều này đảm bảo rằng các lỗ hổng bảo mật được ngăn chặn ngay từ máy cá nhân trước khi có cơ hội xâm nhập vào kho lưu trữ chung trên Cloud.

2. Công cụ quản lý dự án Maven và tệp pom.xml: Đối với dự án Java, Maven đóng vai trò là hệ thống điều phối thực thi và quản lý vòng đời phần mềm. Mọi quy trình từ việc tiếp nhận thư viện đầu vào đến các tiêu chuẩn biên dịch mã nguồn đều được định nghĩa tập trung tại tệp cấu hình pom.xml. Có thể hình dung Maven như một dây chuyền sản xuất tự động, nơi mỗi sản phẩm đầu ra đều phải tuân thủ nghiêm ngặt các thông số kỹ thuật đã được thiết lập sẵn trong bản thiết kế pom.xml

3. Cơ chế Profile (-P): Maven cho phép tạo ra các kịch bản chạy khác nhau gọi là Profile. Ví dụ, khi gõ lệnh kèm tham số -P audit-mode, ta đang ra lệnh cho Maven không build code theo cách thông thường, mà chuyển sang một chế độ đặc biệt để phục vụ riêng cho mục đích kiểm toán bảo mật.

1.4. Tư tưởng thiết kế hệ thống

Để giải quyết bài toán dựa trên các công cụ nền tảng trên, kiến trúc phòng thủ tại máy trạm được thiết kế xoay quanh hai tư tưởng cốt lõi:

1. Kiểm toán và ngăn chặn: Phân tách rõ ràng giữa việc thu thập thông tin phục vụ phân tích và thực thi hành động kiểm soát mã nguồn . Chế độ 'audit' quét toàn diện dự án để nhìn thấy bức tranh tổng thể về rủi ro (chỉ ghi log báo cáo, không chặn build). Ngược lại, chế độ 'enforce' là chốt chặn chủ động, đánh giá mã nguồn dựa trên các rào cản chất lượng (Quality Gate) và lập tức từ chối mọi tác vụ nếu phát hiện rủi ro mới vi phạm.

2. Đóng băng nợ kỹ thuật: Thay vì yêu cầu khắc phục triệt để toàn bộ lỗi cũ, điều mà phi thực tế về mặt thời gian và có thể gây cản trở lớn đối với luồng làm việc. Nhóm khoanh vùng và tạm "bỏ qua" các lỗ hổng đang tồn tại, các lỗ hổng này đã được phát hiện và được cấu hình đích danh để cho qua luồng kiểm soát. Nguyên tắc là phân tích và hiểu rõ về mối đe dọa hiện hữu đối với hệ thống nhưng sẽ cho luồng kiểm soát hoạt động ngay để phát hiện và kiểm soát.

1.5. Mô hình Kiến trúc

Từ tư tưởng thiết kế đó, quy trình kiểm thử tĩnh tại máy trạm - Pha 1 - được chia thành hai luồng thực thi độc lập:

1. Luồng Phát triển hàng ngày: Đây là luồng thực thi chế độ Enforce, hoạt động vô hình và nhúng trực tiếp vào thao tác của lập trình viên thông qua cơ chế pre-commit hooks. Mỗi khi lệnh commit được gọi, kịch bản '.pre-commit-config.yaml' tự động kích hoạt quét bí mật lọt lộ và kiểm tra tĩnh bằng các công cụ SAST. Như đã đề cập, luồng này áp dụng các cơ chế loại trừ để bỏ qua nợ kỹ thuật cũ nhưng đảm bảo code mới luôn sạch.
2. Luồng Kiểm toán tổng thể: Đây là luồng Audit chuyên biệt, dùng để đánh giá sức khỏe toàn diện của dự án. Luồng này được vận hành thông qua file 'audit-scan.bat', kích hoạt profile -P audit-mode trong tệp pom.xml. Khi chạy, nó vô hiệu hóa các rào cản, không bỏ qua lỗi cũ và quét ở chế độ nghiêm ngặt nhất trích xuất báo cáo lỗi đầy đủ vào thư mục audit-reports.

Phần 2. Nền tảng quy trình và các công cụ

Để hiện thực hóa tư tưởng thiết kế phân luồng và đóng băng nợ kỹ thuật, nhóm lựa chọn một chuỗi công cụ dựa trên ba tiêu chí cốt lõi: khả năng chạy độc lập tại local, dễ dàng tích hợp vào Maven, và hơn hết hoạt động miễn phí đối với sinh viên, điều mà thường đi kèm với tính "kiểm soát được" cao. Mỗi công cụ được phân công một vai trò chuyên biệt, tạo thành một hệ thống an ninh nhiều lớp.

2.1. Git và pre-commit framework

Pre-commit framework đóng vai trò là chốt chặn trên máy cá nhân của lập trình viên. Thay vì dựa vào ý thức tự giác chạy các lệnh kiểm tra, framework này can thiệp trực tiếp vào tiến trình của Git.

Bằng cách thiết lập kịch bản cấu hình trung tâm '.pre-commit-config.yaml', mọi sự kiện git commit đều bị tạm giữ để hệ thống tự động kích hoạt tuần tự các công cụ dò quét. Pre-commit đóng vai trò là một chốt chặn thực thi nghiêm ngặt. Chỉ khi mã nguồn vượt qua toàn bộ các tiêu chuẩn kiểm tra an toàn đã thiết lập, hệ thống mới cho phép hoàn tất lệnh commit và chính thức ghi nhận các thay đổi vào lịch sử phiên bản.

2.2. GitLeaks – công cụ rà quét bí mật

Trong phát triển phần mềm, việc vô tình để lộ thông tin nhạy cảm như mật khẩu cơ sở dữ liệu, API keys hay token vào kho lưu trữ là một rủi ro gây thiệt hại trực tiếp về tài chính và uy tín. Gitleaks được tích hợp dựa trên triết lý zero-trust, đóng vai trò là lớp màng lọc nhạy bén với khả năng cập nhật liên tục.

Điểm ưu việt của Gitleaks nằm ở thư viện quy tắc khổng lồ, được đội ngũ phát triển cập nhật liên tục để nhận diện chính xác các cấu trúc định danh mới nhất từ các nhà cung cấp dịch vụ đám mây điển hình như: AWS, Google Cloud, Azure và các nền tảng thanh toán. Khả năng này giúp hệ thống phát hiện ngay lập tức các mã khóa vừa được phát hành, từ đó ngăn chặn rủi ro khai thác trái phép tài khoản hoặc chiếm đoạt tài nguyên đám mây - những kịch bản có thể gây tổn thất kinh tế nghiêm trọng chỉ trong vài giây sau khi lộ lọt.

Tuy nhiên, để đảm bảo tính linh hoạt trong quá trình kiểm thử, nơi lập trình viên đôi khi phải dùng mật khẩu giả để test. Nhóm đã thiết lập một cơ chế ngoại lệ thông qua tệp .gitleaks.toml. Cấu hình [allowlist] cho phép hệ thống định nghĩa rõ ràng các chuỗi an toàn (ví dụ: GITLEAKS_DUMMY_REGEX_123456789), giúp GitLeaks bỏ qua các dữ liệu giả này mà vẫn duy trì khả năng chặn đứng lập tức mọi thông tin nhạy cảm thực sự.

2.3. PMD và SpotBugs

Đối với việc phân tích mã tĩnh, thay vì chỉ sử dụng một công cụ duy nhất, nhóm đã quyết định thiết lập một hệ thống phòng thủ kép bằng việc kết hợp PMD và SpotBugs. Quyết định này xuất phát từ bản chất hoạt động bù trừ lẫn nhau của hai công cụ

2.3.1 PMD

PMD phân tích trực tiếp các tệp văn bản mã nguồn '.java'. Trong quy trình, PMD đóng vai trò là công cụ kiểm duyệt quy chuẩn, chuyên trách việc nhận diện các sai phạm về quy tắc lập trình như mã nguồn dư thừa hoặc các lỗ hổng bảo mật sơ đẳng thông qua cấu trúc văn bản.

Bằng cách tích hợp các bộ quy tắc chuyên biệt trực tiếp vào tệp cấu hình pom.xml, PMD thiết lập một hệ thống cảnh báo đa lớp. Trong đó:

1. security.xml: Đóng vai trò là màng lọc an ninh, chuyên truy vết các mẫu hình mã nguồn nguy hiểm như mã hóa yếu hoặc thiếu sót trong kiểm soát dữ liệu đầu vào.
2. errorprone.xml: Tập trung vào việc loại bỏ các 'nợ kỹ thuật' tiềm tàng như rò rỉ tài nguyên hay các logic sai sót dễ gây đình trệ hệ thống.

Sự kết hợp này mang lại khả năng phản hồi tức thời, giúp đội ngũ không chỉ vá các lỗ hổng bảo mật mà còn nâng cao tính ổn định và độ bền vững của toàn bộ hệ thống.

2.3.2 SpotBugs

Khác với phương pháp phân tích file .java của PMD, SpotBugs thực hiện rà quét trực tiếp trên bytecode .class sau khi biên dịch. Điểm cốt lõi trong kiến trúc bảo mật của nhóm là việc tích hợp FindSecBugs - một thư viện mở rộng chuyên biệt cho an ninh ứng dụng Java.

FindSecBugs cung cấp danh mục hơn 130 loại lỗ hổng dựa trên các tiêu chuẩn quốc tế như OWASP Top 10 và CWE. Công cụ này vận hành dựa trên cơ chế phân tích nhiễm độc, có khả năng truy vết một biến dữ liệu từ điểm tiếp nhận xuyên qua các lớp xử lý trung gian cho đến điểm thực thi cuối cùng. Nhờ vậy, hệ thống có thể nhận diện chính xác các kịch bản tấn công phức tạp như SQL injection, cross-site scripting (XSS) hay path traversal - những lỗ hổng logic ẩn sâu trong luồng thực thi mà các công cụ quét bề mặt không thể truy xuất được.

Sự phối hợp giữa PMD và SpotBugs tích hợp FindSecBugs tạo thành một hệ thống phòng thủ đa tầng. Từ việc chuẩn hóa hình thức mã nguồn đến việc thắt chặt an ninh luồng dữ liệu, đảm bảo dự án microShop được bảo vệ tốt trong hệ sinh thái Maven.

Phần 3. Thực nghiệm quét toàn diện và phân tích

3.1. Tool quét audit: audit-scan.bat

Việc đánh giá toàn diện một dự án đã có thời gian phát triển lâu, cơ sở mã nguồn lớn cần các công cụ phân tích phải hoạt động ở công suất tối đa để rà quét toàn bộ mã nguồn. Tuy nhiên, nếu kích hoạt chế độ này trong luồng phát triển thông thường, hệ thống sẽ lập tức chặn do khối lượng lỗi bị dồn ứ quá lớn. Để giải quyết vấn đề này, Tool kiểm toán tự động audit-scan.bat được thiết kế như một công cụ "một chạm" dành riêng cho mục đích quét toàn diện và phân tích.

Khi được kích hoạt thông qua tệp audit-scan.bat, hệ thống sẽ điều phối Gitleaks và Maven thực thi kèm tham số -P audit-mode. Tham số này gọi kịch bản audit-mode đã được cấu hình trong file ‘pom.xml’ cho phép vượt qua các rào cản ngắt biên dịch thông thường và nâng cường độ quét cũng như báo tất cả các rủi ro mà các công cụ dò quét có thể thấy được.

Lý do chỉ audit-mode mới để các công cụ quét và báo tất cả những gì nó thấy bởi vì phần lớn các báo cáo trả về thường có mức độ rủi ro thấp và mức độ false positive cao. Những thông tin này sẽ hữu ích đối với việc phân tích nhưng sẽ là ác mộng nếu lập trình viên bị bắt phải nghe theo cả đống những báo cáo chất lượng thấp này.

Tại chế độ audit-mode, hệ thống sử dụng tệp cấu hình spotbugs-audit-allow-all.xml -một file miễn trừ giả để chỉ thị rằng không có miễn trừ nào ở đây cả, từ đó đảm bảo quá trình rà quét được diễn ra xuyên suốt trên toàn bộ khối lượng mã nguồn đồ sộ của dự án. Thay vì hiển thị dữ liệu rời rạc trên giao diện dòng lệnh, toàn bộ kết quả kiểm toán được định tuyến và kết xuất thành các báo cáo tĩnh trong thư mục audit-reports. Các tệp tin ‘gitleaks-audit.json’, ‘pmd.xml’, và ‘spotbugs.xml’ cung cấp một cái nhìn toàn cảnh về 'sức khỏe bảo mật' của dự án, tạo tiền đề để đội ngũ bảo bật phân tích, thiết lập mức cơ sở (ranh giới rủi ro chấp nhận được của hệ thống) và hoạch định lộ trình khắc phục nợ kỹ thuật một cách khoa học.

3.2. Phân tích thực tế mã nguồn

Kết quả trích xuất từ tool audit-scan đã cung cấp một bức tranh toàn cảnh về mức độ rủi ro của dự án microShop trước khi chính thức áp dụng quy trình DevSecOps:

3.2.1 GitLeaks

Kết quả: an toàn tuyệt đối (0 vi phạm).

- Báo cáo 'gitleaks-audit.json' không ghi nhận bất kỳ dấu vết nào của việc lộ lọt thông tin nhạy cảm. Điều này chứng tỏ mã nguồn hiện tại hoàn toàn sạch bóng các mật khẩu, token hay API keys bị nhúng trực tiếp ở định dạng văn bản rõ. Tuy nhiên, đây chỉ là bề mặt của vấn đề bảo mật.

3.2.2 PMD

Kết quả: Ghi nhận tổng cộng 377 cảnh báo về chất lượng và cấu trúc mã nguồn thông qua báo cáo 'pmd.xml'.

Phân tích sâu hơn, hệ thống chia các vi phạm này thành 2 nhóm ưu tiên chính:

1. Mức độ thấp (Priority 5 - 261 lỗi): Chiếm tỷ trọng lớn nhất (hơn 69%) là các lỗi thuộc nhóm DataflowAnomalyAnalysis (Dị thường luồng dữ liệu). Đây là các lỗi khai báo biến thừa, ghi đè biến không cần thiết (UR/DU/DD anomalies), làm lãng phí bộ nhớ và giảm hiệu năng thực thi của các lớp Controller và DAO.

2. Mức độ trung bình/cao (Priority 3 - 116 lỗi): Tập trung vào các vi phạm nguy hiểm hơn đối với tính toàn vẹn của ứng dụng, tiêu biểu như NonSerializableClass (61 lỗi) và MissingSerialVersionUID (18 lỗi) – tiềm ẩn nguy cơ lỗi khi hệ thống cần tuần tự hóa dữ liệu (Serialization) trên các Session. Ngoài ra, phát hiện 19 lỗi AvoidLiteralsInIfCondition, minh chứng cho việc lập trình viên đang lạm dụng mã hóa cứng (hardcode) các hằng số logic, gây khó khăn cho việc bảo trì.

3.2.3 SpotBugs + plugin FindSecBugs:

Kết quả: Phát hiện 89 lỗi nghiêm trọng thông qua luồng quét spotbugs.xml.

Phân tích rủi ro lõi: Trong khi PMD chỉ dừng ở mức độ "nợ kỹ thuật" bề mặt, SpotBugs thực sự đã tìm trúng các lỗ hổng bảo mật cốt lõi. Trong số 89 lỗi này, phần lớn được phân loại ở mức độ rủi ro Trung bình đến Cao (Medium to High Severity), tập trung vào các nhóm nguy hiểm nhất:

1. Nhóm lỗ hổng tiêm nhiễm (Injection): Việc truy vết các biến qua các luồng thực thi đã phát hiện các điểm kết nối Cơ sở dữ liệu lỏng lẻo tại tầng DAO, mở ra nguy cơ tấn công SQL Injection.
2. Nhóm quản lý tài nguyên: Các lỗi quên đóng kết nối Database hoặc Input/Output Stream tiềm ẩn rủi ro cạn kiệt tài nguyên khiến hệ thống ngưng trệ.
3. Nhóm quản lý xác thực: Các dấu hiệu cảnh báo về việc nhúng cấu hình mật khẩu hoặc thiếu cơ chế kiểm soát mã hóa đủ mạnh.

- Kết luận tổng thể phân tích từ audit: Các số liệu thống kê trên (tổng cộng 466 cảnh báo và lỗi) đóng vai trò là thước đo cơ sở vững chắc. Nó không chỉ phản ánh đúng sức khỏe thực trạng của một hệ thống cũ nhiều nợ kỹ thuật như microShop, mà còn là minh chứng không thể chối cãi cho tính cấp thiết của việc phải có quy trình phát triển phần mềm bảo mật.

3.3. Khoanh vùng nợ kỹ thuật và thiết lập baseline

Trong thực tế, việc yêu cầu đội ngũ khắc phục ngay lập tức 377 cảnh báo PMD và 89 lỗi SpotBugs là một rào cản phi thực tế, gây ách tắc toàn bộ tiến độ dự án. Do đó, nhóm áp dụng chiến lược khoanh vùng nợ kỹ thuật để thiết lập một mức cơ sở. Chiến lược này tuân theo nguyên lý cốt lõi tạm bỏ qua lỗi cũ để dự án tiếp tục vận hành, nhưng tuyệt đối ngăn chặn việc phát sinh lỗi mới.

Để thực thi nguyên lý này, nhóm đã đối chiếu danh sách từ báo cáo audit và định nghĩa các lỗi đó vào các tập tin loại trừ chuyên biệt trong thư mục exclude/ (gồm pmd-exclude.properties và spotbugs-exclude.xml). Điểm tinh tế của kiến trúc này là hệ thống không hề tắt bỏ các quy tắc bảo mật. Thay vào đó, nó "trói chặt" từng lỗi cũ vào đúng tọa độ của nó - chỉ đích danh Class, Method, hoặc Field vi phạm.

Về cơ bản, cơ chế thiết lập mức cơ sở đã giải quyết được bài toán hóc búa nhất của việc tích hợp bảo mật là dung hòa giữa tính an toàn tối đa và duy trì nhịp độ phát triển liên tục.

Phần 4. Triển khai quy trình

Sau khi bức tranh toàn cảnh về nợ kỹ thuật đã được làm rõ và khoanh vùng an toàn thông qua audit-scan, hệ thống sẽ được triển khai phòng thủ chủ động. Trọng tâm của giai đoạn này không nằm ở việc công cụ quét mạnh đến đâu, mà nằm ở trải nghiệm thực thi của lập trình viên. Một quy trình bảo mật tốt là quy trình trong suốt trong góc nhìn của dev, tự động và không đòi hỏi cấu hình thủ công phức tạp.

4.1. Tool cài đặt môi trường một chạm setup-env.bat

Thách thức lớn nhất khi triển khai DevSecOps cho đội nhóm là rủi ro sai lệch môi trường. Nếu yêu cầu mỗi lập trình viên phải tự đọc tài liệu, tự cài đặt Python, cấu hình GitLeaks và gắn pre-commit hooks bằng tay, tỷ lệ xảy ra sai sót hoặc cố tình bỏ qua là rất cao.

Để xử lý rủi ro này, nhóm thiết kế công cụ 'setup-env.bat'. Khi một lập trình viên bắt đầu làm việc với quy trình bảo mật mới, việc duy nhất dev cần làm là chạy lệnh setup-env.bat và để tool lo tất cả việc setup môi trường.

Kịch bản sẽ tự động quét hệ thống để kiểm tra các phần mềm nền tảng như Git, JDK, Python, tải xuống các gói phụ thuộc cần thiết và tự động thực thi lệnh pre-commit install. Lệnh này sẽ âm thầm gắn kịch bản kiểm thử vào lõi của Git trên máy cá nhân. Nhờ đó, việc chuẩn bị một môi trường code an toàn trở nên hoàn toàn trong suốt và đồng nhất trên toàn bộ các máy trạm của nhóm.

4.2. Thiết lập Quality Gate

Ngưỡng chất lượng là thước đo để quyết định một đoạn mã có được phép đi tiếp hay không. Dựa trên các cấu hình trong tệp 'pom.xml', nhóm đã thiết lập các ngưỡng chặt chẽ:

1. Đối với PMD: Ngưỡng đánh thất bại được đặt ở mức độ 2. Điều này có nghĩa là công cụ sẽ bỏ qua các lỗi cảnh báo phong cách nhỏ mức 3, 4, 5 để tránh làm phiền lập trình viên, nhưng sẽ lập tức ngắt quá trình biên dịch nếu phát hiện lỗi ở mức độ Blocker (mức 1) hoặc Critical (mức 2).
2. Đối với SpotBugs: Tham số nỗ lực quét 'Effort' được đẩy lên mức Max để rà soát sâu nhất có thể, đi kèm với ngưỡng cảnh báo 'threshold' ở mức Medium. Bất kỳ rủi ro bảo mật nào từ mức trung bình trở lên (như rò rỉ tài nguyên hay các biến thể của Injection) đều bị đánh dấu là vi phạm nghiêm trọng và kích hoạt cơ chế failOnError.

Phần 5. Tổng kết

Việc tích hợp thành công các chốt chặn bảo mật vào dự án cũ như microShop đã minh chứng cho tính khả thi và hiệu quả của mô hình DevSecOps ngay từ máy trạm cá nhân của lập trình viên. Thay vì áp đặt các quy tắc an ninh một cách cứng nhắc làm đình trệ tiến độ dự án, nhóm đã giải quyết trọn vẹn bài toán vận hành thông qua kiến trúc phân luồng kết hợp với chiến lược ”ghi nợ kỹ thuật”.

Thông qua việc tự động hóa hoàn toàn khâu chuẩn bị môi trường bằng kịch bản ‘setup-env.bat’, quy trình bảo mật đã trở nên trong suốt đối với lập trình viên. Việc nhúng các công cụ phân tích tĩnh mạnh mẽ gồm GitLeaks, PMD, SpotBugs plugin FindSecBugs trực tiếp vào luồng thao tác thông qua pre-commit hooks đã giúp hiện thực hóa triết lý Shift-Left Security - đẩy các bước kiểm tra an toàn về giai đoạn sớm nhất có thể.

Với các ngưỡng chất lượng được cấu hình nghiêm ngặt, hệ thống hiện tại đảm bảo một nguyên tắc: Chấp nhận sống chung và kiểm soát nợ kỹ thuật trong quá khứ, nhưng tuyệt đối từ chối và tiêu diệt mọi lỗ hổng bảo mật sinh ra trong tương lai. Quy trình này không chỉ tạo ra một phòng tuyến vững chắc bảo vệ mã nguồn trước khi được đẩy lên kho lưu trữ chung, mà còn góp phần định hình một văn hóa lập trình an toàn, bền vững cho toàn bộ đội ngũ phát triển.